

Ambiguity and EBNF

Programming Languages Intro

Dr. Igor Steinmacher

Ambiguous Grammar

- A grammar is *ambiguous* if one of its strings has two or more different parse trees to derive them
E.g., Grammar G_1 above is unambiguous.
- C, C++, and Java have many operators and precedence levels

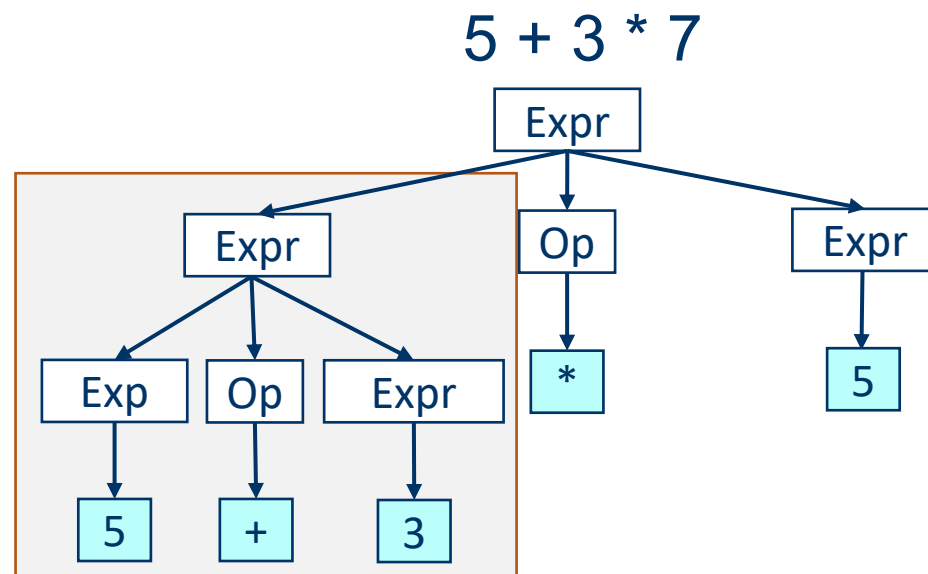
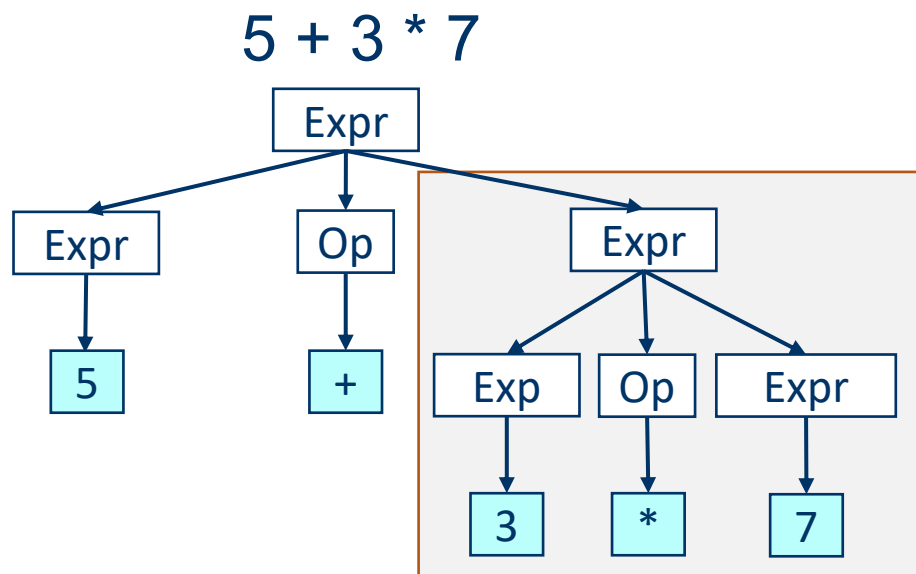
Ambiguous Grammar

```
Expr → Expr Op Expr | ( Expr ) | 0 | 1 | ... | 9 |  
Op → + | - | * | / | % | **
```

- This Grammar is equivalent to our previous one
 - Same language can be derived from both
- This is simpler!
- This is **AMBIGUOUS!!!**

Ambiguous Grammar

$\text{Expr} \rightarrow \text{Expr Op Expr} \mid (\text{Expr}) \mid 0 \mid 1 \mid \dots \mid 9 \mid$
 $\text{Op} \rightarrow + \mid - \mid * \mid / \mid \% \mid **$



The Dangling “Else” Problem

```
IfStat → if ( Expression ) Stat |  
        if ( Expression ) Stat else Stat  
Stat   → Assignment | IfStatement | ...
```

With which ‘if’ does the following ‘else’
associate

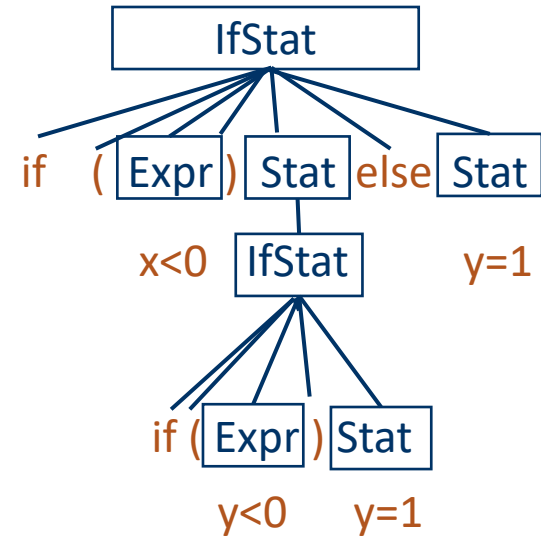
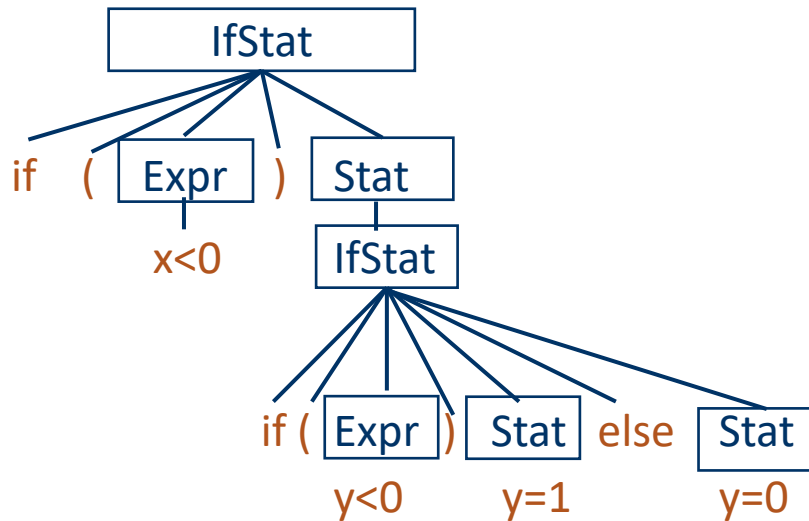
```
if (x < 0)  
    if (y < 0)    y = 1;  
else y = 0;
```

Answer: *either one!*

IfStat $\rightarrow$ `if ( Expr ) Stat` |
 `if ( Expr ) Stat else Stat`
 Stat $\rightarrow$ Assignment | IfStat | ...

```

if ( x < 0 )
    if ( y < 0 )    y = 1;
else y = 0;
  
```



How to deal with the problem

1. Algol 60, C, C++

associate each else with closest if; use {} or begin...end to override.
Informally, in C: else is paired with the closest if

2. Algol 68, Modula, Ada

use explicit delimiter to end every conditional (e.g., if...fi)

3. Java

rewrite the grammar to limit what can appear in a conditional:

`IfThenStatement` → `if ( Expression ) Statement`

`IfThenElseStatement` → `if ( Expression ) StatementNoShortIf else Statement`

`StatementNoShortIf` includes anything except `IfThenStatement`

Extended BNF (EBNF)

- BNF:
 - Recursion for iteration
 - Non-terminals for grouping
- EBNF: additional metacharacters
 - [] for an optional list; pick none or one
 - { } for a series of zero or more
 - () for a list, must choose one

Extended BNF (EBNF)

- `[ ]` for an optional list; pick none or one

```
IfStat → if ( Expr ) Stat | if ( Expr ) Stat else Stat
```

```
IfStat → if ( Expr ) Stat [ else Stat ]
```

- `( )` for a list, must choose one

```
Expr → Expr + Term | Expr - Term | Term
```

```
Expr → Expr ( + | - ) Term | Term
```

- `{ }` for a series of zero or more

```
Integer → Digit | Digit Integer
```

```
Integer → Digit {Integer}
```

- `{ }+` for a series of ONE or more

Grammar of Clite

A Version of C

Program ⇒ int main () { Declarations Statements }
Declarations ⇒ { Declaration }
Declaration ⇒ Type Identifier ;
Type ⇒ int | bool | float | char
Statements ⇒ { Statement }
Statement ⇒ ; | Block | Assignment | IfStatement | WhileStatement
Block ⇒ { Statements }
Assignment ⇒ Identifier = Expression ;
IfStatement ⇒ if (Expression) Statement [else Statement]

WhileStatement ⇒ while (Expression) Statement
Expression ⇒ Conjunction { || Conjunction }
Conjunction ⇒ Equality { && Equality }

Equality ⇒ Relation [EquOp Relation]
EquOp ⇒ == | !=
Relation ⇒ Addition [RelOp Addition]

RelOp ⇒ < | <= | > | >=

Addition ⇒ Term { AddOp Term }
AddOp ⇒ + | -
Term ⇒ Factor { MulOp Factor }
MulOp ⇒ * | / | %
Factor ⇒ [UnaryOp] Primary
UnaryOp ⇒ - | !
Primary ⇒ Identifier | IntLit | FloatLit | (Expression)

Identifier => Letter { Letter | Digit }
Letter => a | b | ... | z | A | B | ... | Z
Digit => 0 | 1 | ... | 9
Literal => Integer | Boolean | Float | Char
Integer => Digit { Digit }
Boolean => true | false
Float => Integer.Integer
Char => 'ASCIICChar'

Clite grammar misses

- Comments
- Whitespace
- Distinguishing one token $\leq$ from two tokens $< \quad =$
- Distinguishing identifiers from keywords like if
- These issues are addressed by identifying two levels:
 - lexical level
 - syntactic level

**See you in our next
Lesson!**